# /telemetry

OpenTelemetry instrumentation: /telemetry help|on|off|setup|counters|logs []

## Overview

`/telemetry` controls ragent's OpenTelemetry export and displays metric counter help. Turning it on starts OTLP export of **both** metrics and logs; `setup` opens a dialog for the endpoint, protocol, export timing, and the log level; `counters` prints the catalogue of metric names the instrumentation records, grouped into tables and followed by the state of the logs signal; `logs` shows or changes the OTLP **logs** export level.

## Syntax

```
/telemetry
/telemetry help
/telemetry on
/telemetry off
/telemetry setup
/telemetry counters
/telemetry logs
/telemetry logs warn
```

## Options / Subcommands

| Form | Description |
|---|---|
| `help` | Show the subcommand table |
| `on` | Enable OpenTelemetry export (metrics **and** logs) |
| `off` | Disable OpenTelemetry export (metrics **and** logs) |
| `setup` | Open the telemetry setup dialog |
| `counters` | Print the metric counter catalogue and the log-export state |
| `logs` | Report whether OTLP log export is on, and at which level |
| `logs <level>` | Set the log-export level (`all`, `trace`, `debug`, `info`, `warn`, `error`) |

There is no separate switch for the logs signal: `/telemetry on` starts metric and log export together and `/telemetry off` stops both. `/telemetry logs on` and `/telemetry logs off` are answered with a pointer to that, so an old habit gets an explanation rather than silence.

The setup dialog fields are: endpoint, protocol, export interval, export timeout, internal port, and logs level (prefilled with the current level, or `info` - an empty field keeps the default rather than turning export off, which is what `/telemetry on|off` does). Saving the dialog turns telemetry - and therefore log export - on.

## Log export

Ragent ships OTLP **logs** whenever telemetry is on. The `tracing` events ragent already emits are batched to the same collector as the metrics, in addition to the local sink - the TUI log panel, or stderr in a head-

less/server run - which is never redirected and keeps showing everything at the level `--log-level` selected.

- The `telemetry.otel.logs.enabled` key in `ragent.json` is written by `/telemetry on|off` alongside `telemetry.otel.enabled`; setting it by hand is only needed for a hand-edited config. Changing the level while telemetry is off records the level and tells you that `/telemetry on` is required.
- Telemetry settings live in the **user-global** `ragent.json` (`$XDG_CONFIG_HOME/ragent/ragent.json`, normally `~/.config/ragent/ragent.json`). A project `.ragent/ragent.json` cannot hold them: the repo-content guard discards that file's whole `telemetry` block so a repository can neither redirect the export nor switch it on, so a change written there would never apply. A key put in a project config by hand is dropped on load for the same reason.
- The level is a floor for the **remote** export only: at `logs warn`, `info` and `debug` records still appear in the local log.
- **Per-frame provider stream diagnostics are never exported.** A completed provider response logs one line per frame (`Ollama Cloud: done frame received`, and a second `Ollama Cloud: tool_calls found in done frame` when the response batches its tool calls into its final frame). Those repeat once per request, so over a session they are pure noise in a collector, and the same information already reaches it through the usage metrics. They stay in the local sink, where they are useful while debugging a provider, and are withheld from the remote export at every `logs` level (`SuppressNoiseEvents` in `ragent_telemetry::logs`).
- `logs all` sets the floor below `TRACE`, so every record the layer observes is shipped. It is the most verbose setting; the local log is unchanged.
- **The log window exports at each entry's own severity.** Every line the TUI log window records is also re-emitted to `tracing` as a record carrying the attributes `log_window = true` and `log_window_level` (the entry's own severity: `tool`, `info`, `warn` or `error`). **A tool call reaches the collector this way and no other**, and it travels at `INFO`, so it arrives at the default `--log-level info` and the default `logs info` with no extra configuration. Use `logs debug` (or `logs trace` / `logs all`) only if you also want ragent's own diagnostic churn.
- **The export does not wait for the panel to be open.** A session normally runs with the log panel closed, so gating the re-emit on the panel being visible meant a collector received only the events that carry a native `tracing` record of their own: `Bash` logs `Executing bash command` on every call, while `Edit`, `Write`, `Grep` and `read` log nothing. From the collector that looked like "only Bash tool calls are exported". The re-emit now runs for every entry regardless of whether the panel is visible; the panel and its spool file are still only written while it is open.
- **Why the entry's own severity.** An earlier revision emitted every re-emit at a fixed `DEBUG`. That kept the local and remote streams disjoint, but it placed the transport below *both* default floors: `--log-level` (then `warn`) and `logs.level` (`info`). A tool call therefore reached a collector only when the operator also raised `--log-level`, which is the opposite of what a telemetry feature should require - in practice the logs signal carried no tool-call activity at all. `--log-level` now defaults to `info`, and each entry keeps its own severity on the wire.
- **Duplication is prevented at the call site, not by a quiet level.** A few panel lines are *copies* of something ragent already logged through `tracing`: the drain of `TuiTracingLayer` records (startup, provider, code-index, MCP and export-health messages) and the mirrored child-stderr drain. Those go through a deferred push that records the line in the panel and the spool and never re-emits it, so the collector sees the original once and the copy never. Every other entry - the ones with no native event behind them, including all tool calls - is re-emitted exactly once.
- **Agent-event-bus lines are exported at their own severity.** The panel lines that come from the agent event bus rather than from `tracing` - `Context compression started`, `response`

`finished (...)`, `agent_complete signalled`, the compaction notice and every `agent notice: ...` - are re-emitted as `INF`, `WRN` or `ERR` matching the marker the panel shows, so a collector receives them under the same severity the operator sees. None of them has a native event behind it, so none of them is a duplicate.

- **Mirrored child stderr stays local.** A subprocess's stderr appears in the panel as a `warn` line and is not exported: it is another process's output, not one of ragent's own events. An MCP server launched through `npx` alone prints hundreds of banner, deprecation and readiness lines at startup, and exporting them would bury the actual signal.
- `--log-level all` is accepted as a spelling of `trace`, so one flag makes the process record every severity for both the local log and the export.
- Records go to the `/v1/logs` sibling of the configured endpoint, on the same protocol, timeout and TLS rules as the metrics.
- Bodies are redacted through the shared credential redactor and attributes through the metric sensitive-data guard; a body over 16 KiB is truncated.
- The change is applied to the running process immediately - no restart. If a reconfigure fails, the configuration is still saved and takes effect on the next start.

## Examples

```
From: /telemetry on
OpenTelemetry export enabled; spans export to the configured endpoint.
```

```
From: /telemetry off
OpenTelemetry export disabled.
```

```
From: /telemetry setup
Opens the ProviderSetupStep::TelemetrySetup dialog with endpoint, protocol,
export interval, export timeout, and internal port fields.
```

```
From: /telemetry logs
OTLP log export is **on** (`telemetry.otel.logs.level` = `info`).
```

```
From: /telemetry logs warn
[ok] OTLP log export enabled at level `warn`.
Records are batched to the configured collector alongside the metrics; the local
log panel keeps showing everything.
```

```
From: /telemetry counters
## Usage
| Metric | Description |
| ragent.llm.requests | ...
| ragent.sessions.active | ...
| ragent.sessions.total | ...
| ragent.messages.user | ...
| ragent.tool.invocations | ...
| ragent.agents.active | ...
| ragent.agents.completed | ...
| ragent.subagent.spawns | ...
| ragent.team.members | ...
```

```
From: /telemetry counters (performance section)
## Performance
```

```
| ragent.llm.duration | ...
| ragent.llm.time_to_first_token | ...
| ragent.tool.duration | ...
| ragent.agent_loop.duration | ...
| ragent.agent_loop.iterations | ...
| ragent.session.duration | ...
| ragent.tool.permission_wait | ...
```

```
From: /telemetry counters (cost section)
## Cost
| ragent.tokens.input | ...
| ragent.tokens.output | ...
| ragent.tokens.cache_read | ...
| ragent.tokens.cache_write | ...
| ragent.cost.estimated | ...
| ragent.cost.session | ...
(8 cost metrics in total)
```

## Output

`on`/`off` print a one-line confirmation. `setup` opens a dialog whose values persist to the config. `counters` prints three grouped metric tables: **Usage** (9 metrics: requests, active/total sessions, user messages, tool invocations, active/completed agents, subagent spawns, team members), **Performance** (7 metrics: LLM duration, time-to-first-token, tool duration, agent-loop duration and iterations, session duration, tool permission wait), and **Cost** (8 metrics: input/output/cache tokens, estimated and session cost), then a **Logs** line reporting whether OTLP log export is on and at which level. `logs` prints the state, or the confirmation of the change.

## Related

- `/telemetry_panel` - display-only side panel fed by these counters
- `/telemetry setup` values persist in the user-global `ragent.json`